Companies feature PRD

A bounded, testable specification for the Next.js developer assessment.

Build a small Companies workflow that demonstrates correct data handling, URL state, concurrency, typing and accessibility. The strongest submission is a complete, explainable implementation of the requested scope. Visual polish beyond basic usability does not earn points.

Purpose and scope

User: a B2B sales or marketing operator who needs to find a company, inspect its details and create or edit a record without overwriting another person's changes.

Outcome: the operator can search, filter, sort and paginate companies; share a link to an open drawer; save validated changes; and recover from a version conflict while keeping their draft.

Assessment requirements: Next.js 16 App Router, React 19, TypeScript strict mode, Tailwind, TanStack Query, a handwritten typed client, four mock API endpoints and the specified tests. The practical work is budgeted at about four hours; the full assessment at four to five hours.

Separate production context from this build

The background product uses FastAPI, OpenAPI-generated clients, hosted identity, organization permissions and tenant isolation. Part 2 explicitly substitutes local Route Handlers, a small typed client, fake sign-in and an in-memory array. Do not build a Python backend, a real identity integration or a tenant-management system. In the real product, permissions and isolation remain API responsibilities.

Definition of success

Area	Observable outcome
Correctness	Every required workflow works, especially stale-version rejection, draft recovery, URL restoration and bearer auth.
Code quality	Clear ownership of URL, server and draft state; strict types; small modules; no dead code, explicit any or @ts-ignore.
Judgement	README explains decisions, scope limits and realistic next steps. No unnecessary features.
Communication	Nine short reasoned answers, reproducible run instructions and meaningful Git commits.

Explicit exclusions

No delete, bulk actions, CRM extras, dashboards, real OAuth, database, deployment requirement, OpenAPI generation pipeline, elaborate design system or mandatory automated E2E suite. The assessment permits AI and documentation, but the candidate must understand and defend every submitted line.

Assessment source: nextjs-developer-test.pdf, supplied by the candidate. Page references below refer to its four pages.

Functional requirements and acceptance criteria

All rows are required by the assessment. Concrete URL syntax, defaults and recovery interactions are proposed implementation choices.

ID	Requirement	Acceptance criteria
F01	Company list	/companies shows name, domain, status and created date. Initial seed is 40 companies; page size is 20, yielding two initial pages.
F02	Search and status	Search matches name or domain; status has all five values plus All. Applying either resets page to 1 and changes the URL. A no-match result offers Clear filters.
F03	Sort and pagination	Name and created date toggle ascending/descending. Previous/Next respect total. Filter, sort and page survive refresh and browser Back/Forward.
F04	Detail drawer	A row click opens the existing company in a side drawer over the mounted list. A company ID in the URL restores the same drawer on direct load or refresh.
F05	Create	Create company opens the same drawer in create mode. Valid submission uses POST; success returns the new record and refreshes relevant list results.
F06	Edit and conflict	Form edits name, domain and status. PATCH sends its captured base version. A 409 explains the conflict and offers latest-data reload with draft preservation and explicit review before another save.
F07	States and validation	List and drawer each have loading, retryable error and empty/missing states. API 422 errors map to fields or a form summary. A filter change does not remount the table.
F08	Fake authentication	Sign-in stores a non-empty demo token and all four API calls attach it as a bearer header. Missing/invalid bearer shape receives 401; a client 401 clears the session and routes to sign-in.
F09	Dutch and English	NL/EN changes visible feature labels and date formatting. Include labels, statuses, buttons and validation/conflict/error messages. Company data is unchanged.
F10	Keyboard and announcements	Semantic table, keyboard-operable row action and sort controls, visible focus, labelled fields, trapped drawer focus, Escape close, focus return and announced errors.

Suggested URL contract

```
/companies?q=acme&status=qualified&sort=-created_at&page=2
/companies?q=acme&sort=name&page=1&company=company-001
/companies?sort=name&page=1&drawer=create
```

Use page=1, sort=name, empty q and no status filter as defaults. Detail and create are mutually exclusive. The query parameters company and drawer describe UI state; they are never included in the list API query or list cache key. Opening or closing a drawer preserves all list parameters.

API contract

Required base path: /api/v1/companies. All supported operations require Authorization: Bearer <non-empty token> and incur an artificial 300-800 ms delay, including error responses. Use Next.js Route Handlers and a shared in-memory array seeded once with 40 records.

Operation	Request	Required response
GET collection	limit, offset, q, status, sort	200: { items, total, limit, offset }. q searches name/domain; status filters; sort accepts name, -name, created_at or -created_at.
GET /{id}	Path ID	200: Company. 404 if missing.
POST collection	{ name, domain?, status? }	201: Company with version 1. 422 if name is empty.
PATCH /{id}	{ if_version, name?, domain?, status? }	200: updated Company with version + 1. 409 if version differs. 404 if missing.

Company data model

```
type Company = {
  id: string;
  name: string;
  domain: string | null;
  status: "new" | "researching" | "contacted"
    | "qualified" | "rejected";
  version: number;
  created_at: string; // ISO 8601
  updated_at: string; // ISO 8601
};
```

Decisions for unspecified behavior

Decision	Proposed contract
Input normalization	Trim name and reject empty/whitespace names in both API and form. Optional blank domain becomes null; an omitted PATCH field is unchanged. Domain is not subject to extra DNS/URL validation.
Creation defaults	Omitted domain becomes null; omitted status becomes new. Generate string ID and ISO timestamps. PATCH preserves id and created_at and updates updated_at.
List rules	Case-insensitive substring search; null domain never matches. Apply search/status, calculate total, sort, then slice. Use ID as a deterministic tie-breaker.
Invalid input	Invalid JSON returns 400. Invalid field types/enums, nonpositive or non-integer if_version, and invalid supplied list parameters return 422 with a typed error body. Omitted limit/offset/sort default to 20/0/name.
Pagination	API accepts a positive integer limit and nonnegative integer offset. UI always requests limit=20 and offset=(page-1)*20. Invalid URL values normalize to safe defaults.

These choices fill gaps in the brief; they are not additional employer requirements. For an out-of-range UI page, show an empty page with a route back to page 1 rather than failing.

Interaction and state behavior

Surface	Required behavior
List initial load	Keep page heading and controls available; render a table skeleton/status with an accessible loading announcement.
List refetch	Keep the same table instance and previous rows visible with Updating results and aria-busy. Indicate that results are stale until the new request finishes.
List error	Show an inline message and Retry. If earlier rows exist, retain and label them; otherwise show the error in the table region. Avoid retry loops for 4xx errors.
List empty	Distinguish no companies from no filter matches. Offer Create company or Clear filters respectively. Treat an out-of-range page separately.
Drawer load	Show drawer title, close control and its own loading state; list stays mounted underneath.
Drawer error/missing	Network/5xx: inline Retry. 404: Company not found and Close. Create mode starts with an empty form instead of fetching an ID.
Form submission	Disable repeat submission while pending. Keep the draft for network, 422 and 409 failures. On success announce Saved, update cache and close the drawer.
Validation	Show field-level messages and a summary; focus the first invalid input. Connect messages with aria-describedby and set aria-invalid.

Version conflict recovery

1. Open: capture the fetched company as the form baseline, including its version. Draft fields are local state; background query updates must not reset them or silently advance the baseline version.

2. Save: send the captured version with the draft changes. On 409 leave the draft visible and announce: "This company was changed by someone else. Your changes are still here. Reload the latest version to review before saving." Block another save until reload/review.

3. Reload: explicitly fetch the latest company through TanStack Query. Start a new baseline from that result and reapply only fields changed by the user; untouched fields take the latest server values. Keep the original draft snapshot and show latest values for comparison.

4. Review and save: the user reviews the proposed values and explicitly saves against this new baseline version. Never automatically retry a conflicting PATCH or overwrite the draft on query invalidation. Another 409 repeats the same recovery.

Accessibility acceptance

Use a real table with column headers and aria-sort on sortable headers. Provide a native link or button in each row as its keyboard action; row click is an additional convenience. The modal drawer has a labelled dialog, initial focus, a focus trap, Escape handling and focus restoration to the opener (or heading if the opener disappeared). Announce validation, fetch failures, conflicts and save outcomes using appropriate alert/status regions.

Technical requirements and completion gate

ID	Required implementation or evidence
T01	Next.js 16 App Router, React 19, TypeScript strict, Tailwind; no explicit any, @ts-ignore or dead code.
T02	TanStack Query owns all company server state and mutations. No useEffect + fetch loading. Local form drafts and URL control state have separate owners.
T03	A small typed client covers all four operations, typed requests/responses/errors and query option helpers such as listCompaniesOptions(params).
T04	At least component tests for list filtering and 409 recovery, plus a unit test for client or query-key logic.
T05	pnpm lint, pnpm typecheck, pnpm test and pnpm build all pass on the final revision.
D01	Shareable Git repository with meaningful commits and README covering run steps, decisions and why, more-time improvements and conscious omissions.
D02	Answers to all nine Part 1 questions, each a few reasoned sentences. The candidate can explain the submitted implementation.

Source coverage

Source page	Covered here
1	Production context and assessment time budget on PRD page 1; questions 1-5 in the written answers document.
2	Questions 6-9 in the written answers document; API contract, authorization and delay on PRD page 3.
3	Exact data fields on page 3; all nine UI rules on pages 2 and 4; technical rules in the table above.
4	Git/README deliverables, assessment criteria and scope discipline on pages 1 and 5 and in the implementation plan.

Limits to document honestly

The memory store is a single-process demo: data resets on restart, and separate server instances do not share it. Fake auth does not validate identity, expire tokens or enforce organizations. The handwritten typed client imitates a generated client; it is not generated from a schema. No durable cross-navigation draft recovery is required; this plan preserves drafts during conflict recovery while the drawer remains open.

Production upgrades belong in the README discussion: persistent storage with atomic version checks, hosted-identity SDK, server-enforced tenant permissions and an actual generated API client. Do not implement those upgrades inside the four-hour scope.

Planning status

This PRD and its companion documents are preparation artifacts. They specify the implementation and evidence to produce; they do not claim that the application, repository, tests or hiring outcome already exist.

Assessment source: nextjs-developer-test.pdf, supplied by the candidate. Page references below refer to its four pages.